DEPARTMENT OF
COMPUTER SCIENCE & ENGINEERING
Discover. Learn. Empower.

Experiment No.: 6

Student Name: Probal Dhali
UID: 24BAI71013
Branch: AIT-CSE
Section/Group: B
Semester: 5TH
Date of Performance:
Subject Name: Competitive Coding – II
Subject Code: 24CSP-305

Experiment No: 6.1

Aim: To implement an algorithm to find the Lowest Common Ancestor (LCA) of two given nodes in a binary tree.

Objective:

- To understand the concept of the Lowest Common Ancestor in a binary tree.
- To implement a recursive algorithm to find the LCA of two nodes.
- To analyze the time and space complexity of the algorithm.

Problem Statement: Given the root of a binary tree and two nodes, p and q, find their Lowest Common Ancestor (LCA). The LCA is defined as the lowest node in the tree that has both p and q as descendants. A node can be a descendant of itself.

LeetCode Link:
https://leetcode.com/problems/lowest-common-ancestor-of-a-binary-tree/description/?utm_source=chatgpt.com

Procedure/Algorithm/Pseudocode:

1. Start from the root node of the binary tree.
2. If the current node is NULL, return NULL.
3. If the current node is either p or q, return the current node.
4. Recursively search for p and q in the left and right subtrees.
5. If both recursive calls return non-NULL values, the current node is the LCA.
6. If only one recursive call returns a non-NULL value, return that value.
7. Return the final result as the Lowest Common Ancestor.

Code:

```
class Solution {

public:

  TreeNode* lowestCommonAncestor(

    TreeNode* root,

    TreeNode* p,

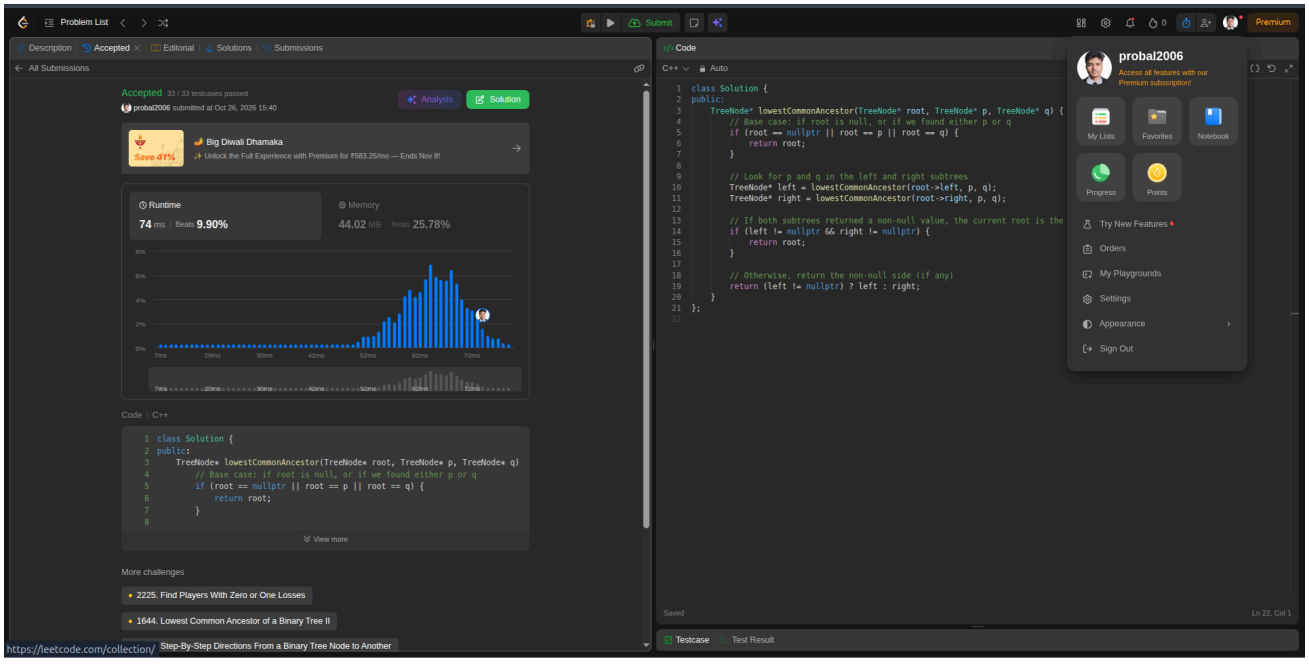
    TreeNode* q

  ) {

    if (root == nullptr || root == p || root == q) {
```

```
        return root;
    }
    TreeNode* left =
        lowestCommonAncestor(root->left, p, q);
    TreeNode* right =
        lowestCommonAncestor(root->right, p, q);
    if (left != nullptr && right != nullptr) {
        return root;
    }
    return (left != nullptr) ? left : right;
  }
};
```

Output:



Observations/Outcome:

- The algorithm successfully finds the Lowest Common Ancestor of two nodes in a binary tree.
- If the two nodes lie in different subtrees, their common ancestor is the current root of those subtrees.
- If one node is an ancestor of the other, the algorithm returns the ancestor node.
- The recursive approach avoids the need to store explicit root-to-node paths.

Time Complexity: $O(N)$

Space Complexity: $O(H)$

Learning outcomes :

- Understood the concept of the Lowest Common Ancestor in a binary tree.
- Learned how to use recursion to traverse a binary tree.
- Implemented a recursive solution for finding the LCA.
- Analyzed the time and auxiliary space complexities of a recursive tree algorithm.

Experiment No: 6.2

Aim: To implement a program to find the in-order successor of a given node in a Binary Search Tree (BST).

Objective: To understand inorder traversal and the concept of finding the next node in sorted order in a BST.

Problem Statement: Given the root of a Binary Search Tree and a node p, find the inorder successor of that node. The inorder successor is the node with the smallest value greater than p->val. If no successor exists, return nullptr.

LeetCode Link: https://leetcode.com/problems/inorder-successor-in-bst/?utm_source=chatgpt.com

Procedure/Algorithm/Pseudocode:

1. Start from the root node.
2. Initialize the successor as nullptr.
3. If p->val is smaller than the current node's value, store the current node as the successor and move left.
4. Otherwise, move to the right subtree.
5. Repeat until the current node becomes nullptr.
6. Return the successor.

Code:

```
class Solution {

public:

   TreeNode* inorderSuccessor(TreeNode* root, TreeNode* p) {

      TreeNode* successor = nullptr;

      while (root != nullptr) {

         if (p->val < root->val) {

            successor = root;

            root = root->left;

         } else {

            root = root->right;

         }

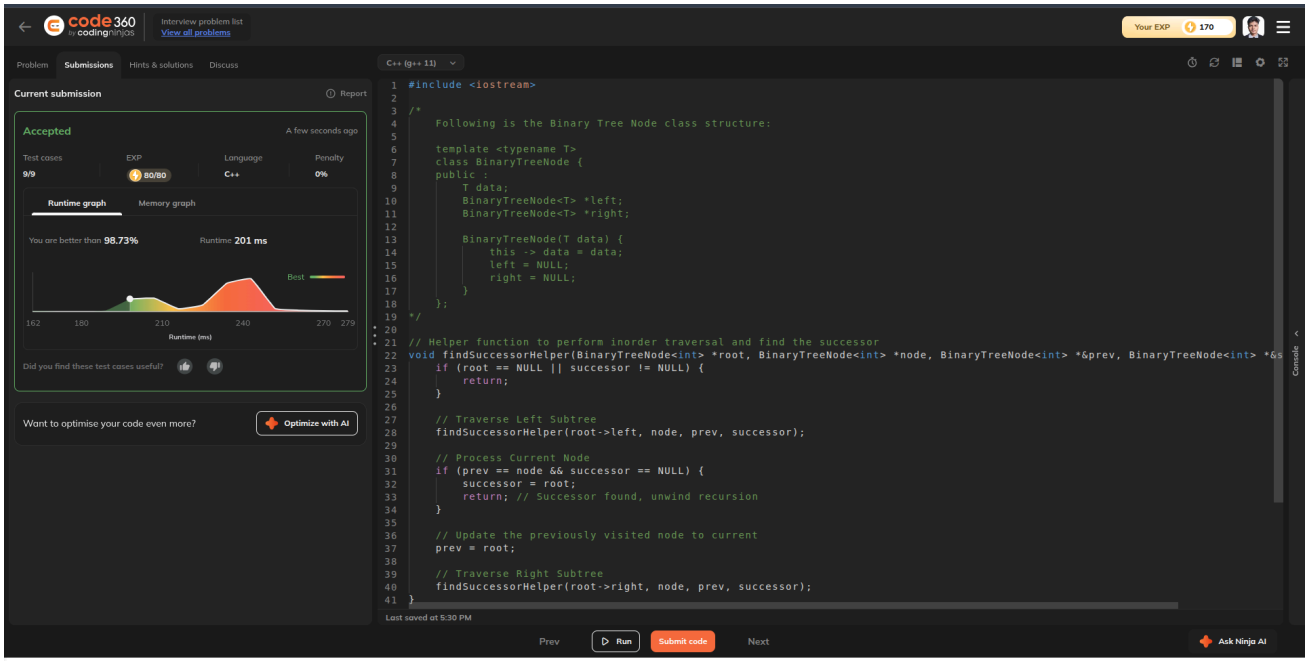
      }

      return successor;

   }

};
```

Output:



Observations/Outcome: The program successfully finds the in-order successor of a given node in a Binary Search Tree. If no greater node exists, it returns nullptr.

Time Complexity: $O(h)$

Space Complexity: $O(1)$

Learning outcomes :

- Understand the concept of inorder successor in a BST.
- Apply BST properties to search efficiently.
- Implement an iterative algorithm in C++.
- Analyze time and space complexity.